

Sisteme de Operare. Laborator 5

Honorius Gâlmeanu
galmeanu@unitbv.ro

March 20, 2023

1 Sistemul de fișiere

Informațiile cuprinse în fișiere se stochează în structuri care se pot înlănțui denumite inode-uri. Fișierul director conține referințe la aceste structuri de tip inod care conțin datele fișierului. Fiecare inod conține un contor de legături, care numără câte legături din structuri de tip director există la el. În mod obișnuit acest contor este egal cu 1, dar, dacă un fișier este înregistrat în mai multe directoare, valoarea contorului crește și ea în mod corespunzător. Operația de ștergere a fișierului dintr-un director (unlink) nu înseamnă că blocurile de date asociate fișierului vor fi șterse. În structura `struct stat`¹ membrul `st_nlink` reprezintă contorul de legături; acest tip de legături direct la inode se numesc **hard links**.

Spre deosebire de **hard link**, legătura simbolică (**symbolic link**) este un fișier special care conține doar o cale - calea către fișierul spre care face referire.

Un inod conține toate informațiile despre fișier: tipul, biții de acces, mărimea fișierului, pointeri către blocurile de date. Majoritatea informațiilor din `struct stat` sunt preluate din inod. Într-un bloc de tip director vor fi stocate doar numerele de inode-uri și numele fișierelor din directorul respectiv. Inode-urile conținute în aceste structuri se vor referi întotdeauna la același sistem de fișiere. Pentru a indica spre fișiere conținute în alte sisteme de fișiere trebuiesc folosite legăturile simbolice.

2 Funcțiile link, unlink, remove și rename

Putem crea o legătură hard către un fișier cu funcția:

```
#include <unistd.h>
int link(const char *oldpath, const char *newpath);
```

Funcția returnează 0 dacă totul este în regulă sau -1 în caz de eroare. Ea creează o nouă intrare într-un bloc de tip director, `newpath`, care va face referire la fișierul existent `oldpath`. Dacă `newpath` există, se va returna eroare. Crearea unei noi intrări și incrementarea contorului de legături trebuie să fie o operație atomică. Numai superuserul poate crea o legătură (hard) nouă care să facă referire la un director. Se evită astfel posibilitatea creării de bucle în sistemul de fișiere.

Pentru a elimina o intrare într-un bloc de tip director se folosește funcția:

```
#include <unistd.h>
int unlink(const char *pathname);
```

La fel, funcția returnează 0 dacă totul este în regulă sau -1 în caz de eroare. Se va șterge fișierul din directorul specificat doar dacă contorul său de legături a ajuns la 0.

Pentru directoare se va utiliza funcția `rmdir` în loc de `unlink`. Putem elimina o legătură și cu funcția `remove`. Pentru un fișier ea este identică cu funcția `unlink`, iar pentru un director, cu funcția `rmdir`:

```
#include <stdio.h>
int remove(const char *pathname);
```

Un fișier sau director poate fi redenumit folosind funcția:

```
#include <stdio.h>
int rename(const char *oldname, const char *newname);
```

¹man 2 stat

3 Funcțiile symlink și readlink

O legătură simbolică se poate crea folosind funcția:

```
#include <unistd.h>
int symlink(const char *target, const char *linkpath);
```

Funcția va crea o legătură denumită linkpath care va arăta către target.

Putem citi conținutul unui fișier de tip legătură simbolică cu funcția:

```
#include <unistd.h>
ssize_t readlink(const char *restrict pathname, char *restrict buf, size_t bufsiz);
```

Funcția va plasa conținutul legăturii simbolice în buf, cu restricționarea numărului de bytes la dimensiunea dată. Rezultatul este numărul de bytes citați sau -1 în caz de eroare.

4 Timpii asociați fișierelor

Există trei timpi asociați unui fișier, descriși de struct stat:

```
struct timespec st_atim; /* Time of last access, ls -lu */
struct timespec st_mtim; /* Time of last modification, ls -l */
struct timespec st_ctim; /* Time of last i-node status change, ls -lc */
```

Timpul ultimului acces se poate utiliza de administratorul de sistem pentru a șterge fișierele care nu au mai fost accesate de foarte mult timp. Timpul ultimei modificări și timpul ultimei schimbări a stării inode-ului se pot utiliza pentru luarea deciziei de arhivare a fișierelor.

Timpul corespunzător ultimului acces și cel al ultimei modificări, pentru un fișier, pot fi modificați cu ajutorul funcțiilor:

```
#include <utime.h>
int utime(const char *filename, const struct utimbuf *times);

#include <sys/time.h>
int utimes(const char *filename, const struct timeval times[2]);
```

Se utilizează structura:

```
struct utimbuf {
    time_t actime;      /* access time */
    time_t modtime;     /* modification time */
};
```

A acțiunile realizate de aceste funcții depind de parametru times și de drepturile necesare:

- dacă times == NULL, cei doi timpi vor fi setați la valoarea timpului curent; pentru aceasta trebuie ca procesul curent să aibă Effective UID egal cu al owner-ului sau să aibă drept de scriere pentru fișierul respectiv;
- dacă times != NULL, cei doi timpi vor fi setați la valorile specificate; Effective UID trebuie să fie egal cu al owner-ului sau procesul să aparțină superuser-ului.

Câmpul st_ctime din structura stat este actualizat automat atunci când este apelată funcția utime.

5 Directoare

Un director poate fi creat cu funcția:

```
#include <sys/stat.h>
int mkdir(const char *pathname, mode_t mode);
```

Funcția va crea un director gol, cu intrările . și .. create automat. Drepturile de acces specificate în mode vor fi modificate de masca procesului. Va trebui să setăm cel puțin unul din biții de execuție pentru a permite accesul la fișierele din director.

Un director gol poate fi șters cu funcția:

```
#include <unistd.h>
int rmdir(const char *pathname);
```

Dacă odată cu acest apel contorul de locații devine 0 și nici un proces nu are deschis directorul, acesta va fi șters.

Directoarele pot fi citite de orice proces are drept de citire pentru ele. Numai kernel-ul poate face scrierea într-un director - scrierea se realizează doar prin apeluri de sistem.

Funcțiile folosite pentru operații cu directoare sunt:

```
#include <sys/types.h>
#include <dirent.h>

DIR *      opendir   (const char *pathname);
struct dirent * readdir (DIR *dp);
void      rewinddir  (DIR *dp);
int        closedir  (DIR *dp);
```

Funcția opendir deschide un stream asociat cu directorul cu numele pathname și setează poziția de citire pe prima înregistrare din director.

Funcția readdir citește următoarea înregistrare din director. Structura citită conține următoarele:

```
struct dirent {
    ino_t      d_ino;      /* Inode number */
    off_t      d_off;      /* Not an offset; see below */
    unsigned short d_reclen; /* Length of this record */
    unsigned char d_type;   /* Type of file; not supported
                             by all filesystem types */
    char        d_name[256]; /* Null-terminated filename */
};
```

Funcția rewinddir setează prima înregistrare drept poziție curentă pentru directorul specificat.

Funcția closedir dealocă structurile folosite de SO pentru parcurgerea directorului anterior deschis.

6 Funcțiile chdir, fchdir și getcwd

Fiecare proces are asignat un director de lucru curent. Nu trebuie confundat cu directorul HOME, care este un atribut al utilizatorului. Putem schimba directorul curent folosind funcțiile:

```
#include <unistd.h>
int chdir(const char *path);
int fchdir(int fd);
```

Pentru a obține directorul curent se apelează una din funcțiile:

```
#include <unistd.h>

char * getcwd      (char *buf, size_t size);
char * getwd       (char *buf);
char * get_current_dir_name(void);
```

Funcția copiază numele directorului curent în buffer-ul specificat. Întoarce un pointer spre numele directorului curent în caz de succes, sau NULL în caz de eroare.

7 Funcțiile chdir, fchdir și getcwd

Directorul de lucru curent al procesului se poate schimba folosind funcțiile:

```
#include <unistd.h>
```

```
int chdir (const char *path);  
int fchdir(int fd);
```

Noul director se poate specifica direct, folosind numele, sau un descriptor de fișier asociat în prealabil cu funcția open.

Pentru a obține directorul de lucru curent se apelează una din funcțiile:

```
#include <unistd.h>
```

```
char *getcwd          (char *buf, size_t size);  
char *getwd           (char *buf);  
char *get_current_dir_name(void);
```

Funcția getcwd scrie calea curentă în buffer-ul buf, alocat cu dimensiunea size. Dacă lungimea căii este mai mare decât dimensiunea buffer-ului, funcția întoarce NULL. Altfel, întoarce un pointer la un șir de caractere conținând calea curentă.

Funcția getwd scrie calea curentă în buffer-ul dat ca parametru, care trebuie să fie alocat de utilizator și să aibă cel puțin dimensiunea PATH_MAX. La fel, întoarce un pointer la un șir ce conține calea curentă.

Funcția get_current_dir_name va realiza singură cu malloc() alocarea numărului de caractere necesar pentru a stoca calea curentă și întoarce buffer-ul alocat și completat.

8 Fișiere device

Un sistem de fișiere este stocat de regulă pe un device. Acest device se poate accesa direct prin fișierele speciale stocate în directorul dev. Device-ul are asociat un identificator, care se compune din două părți: **major** și **minor**.

Pentru un dispozitiv avem doi identificatori: st_dev, care este identificatorul sistemului de fișiere care conține fișierul, respectiv st_rdev, identificatorul device-ului asociat fișierului special. Doar fișierele speciale de tip bloc sau de tip caracter au asociat acest din urmă identificator.

9 Funcțiile sync și fsync

Majoritatea implementărilor UNIX au un buffer cache în kernel prin care se fac transferurile I/O. Când scriem prin funcțiile write într-un fișier, datele vor fi copiate de către kernel în buffer și menținute acolo până când are loc operația fizică de scriere; scrierea se face atunci când buffer-ul desemnat este complet plin. Această modalitate de scriere este numită **delayed write**. Asigurarea consistenței sistemului de fișiere de pe dispozitiv cu conținutul buffer-ului cache (forțarea scrierii chiar dacă buffer-ul nu este plin) se realizează prin apelul următoarelor funcții:

```
#include <unistd.h>  
int fsync (int fd);  
void sync (void);  
int syncfs(int fd);
```

Funcția fsync va forța scrierea datelor buffer-ate către fișierul specificat. Funcția sync va face acest lucru pentru toate sistemele de fișiere montate, iar funcția syncfs va forța scrierea buffer-elor pentru întreg filesystem-ul pe care se află fișierul specificat.

10 Exemple

Pentru a putea compila programele date ca exemplu va fi necesar ca mai întâi să se construiască biblioteca liblab4.a. Acesta se face cu comanda:

Command Line

```
make lib
```

După generarea bibliotecii liblab5.a, se compilează exemplele cu:

Command Line

```
make
```

10.1 Programul unlink

Programul deschide un fișier și face unlink (ștergere). Se observă cum comanda de fișiere nu produce consum de blocuri, în vreme ce crearea unui fișier nou alocă blocuri:

```
$ df /home
$ cp ftw4.c tempfile
$ df /home
$ echo mini >tempfile
$ df /home
$ ./unlink
$ df /home
```

10.2 Programul zap

Programul trunchiază la 0 lungimea unui fișier cu ajutorul funcției open și nu va modifica timpul ultimului acces și nici timpul ultimei modificări. Se exemplifică utilizarea funcției utime. Programul se execută astfel:

```
$ cp unlink tempfile
$ ls -l tempfile
$ ls -lu tempfile
$ date
$ ./zap tempfile
$ ls -l tempfile
$ ls -lu tempfile
$ ls -lc tempfile
```

10.3 Programul ftw4

Programul va realiza traversarea unei ierarhii de fișiere. Va afișa numărul fișierelor pentru fiecare clasă de fișiere. Se exemplifică utilizarea funcțiilor de lucru cu directoare:

```
$ ./ftw4 /usr
```

10.4 Programul cdpwd

Programul va schimba directorul și va afișa directorul de lucru. Se exemplifică utilizarea funcțiilor chdir și getcwd:

```
$ ./cdpwd
$ ls -l /var/spool
```

10.5 Programul devrdev

Programul afișează numerele de device pentru un fișier. Se va executa astfel:

```
$ ./devrdev / /dev/sr0 /dev/tty
```

11 Exerciții

Exercițiul 1

Rulați toate programele prezentate. Asigurați-vă că le-ați înțeles funcționarea. Folosiți-vă de pagina 2 de manual (comanda `man`).

Exercițiul 2

Să se scrie un program care să producă același rezultat ca și comanda `ls` (invocată fără parametri).

Exercițiul 3

Modificați programul `ftw4` astfel încât de fiecare dată când este întâlnit un director să se execute `chdir` la acel director, permițând astfel utilizarea numelor de fișier și nu a căilor complete la apelul funcției `lstat`. După ce toate intrările în director au fost procesate se va executa `chdir("..")`. Comparați timpii de execuție pentru cele două variante folosind comanda de sistem `time`.